

本地 DNS 攻击实验补充说明

1 概述

你需要按照 SEED Labs 实验手册（Homework_3_Manual_DNS_Local.pdf，有中英文版）填写作业模板（Homework_3_template.docx）完成作业 3。这次作业和作业 2 一样使用 Docker 容器，但涉及更多操作。本文档将某些操作进行具体说明，方便你顺利完成作业。此外，默认你已阅读作业 2 帮助文档并完成了作业 2。

2 实验环境设置任务

首先，需要你下载作业文件中的 LabsetupDNS.zip，并将其中的 LabsetupDNS 文件夹解压到 Home 目录下，后续大部分操作需在该目录下执行。你需要先把压缩文件通过共享文件夹传输到虚拟机里，将其放在 Home 目录下，再右键点击，选择 Extract Here。**不要在自己电脑上解压缩再传到虚拟机。**

为了更快地建立容器镜像，同样在 Teams 的 课程 - Homework Files 文件夹中提供了对应压缩包：dns-lab-delta.tar.xz，其 md5 值为 197d3a28c12a0023a4df33e5fd3c63d8。下载该压缩包后，通过共享文件夹把它传到虚拟机，再复制到 LabsetupDNS 文件夹（不要解压缩）。

在 LabsetupDNS 文件夹里打开终端窗口，首先运行 `docker images` 指令查看已安装的镜像：

```
[03/23/26]seed@VM:~/Labsetup$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
seed-image-attacker-csrf	latest	0801b817246a	8 hours ago	261MB
seed-image-mysql-csrf	latest	c6dd466ca348	8 hours ago	545MB
seed-image-www-csrf	latest	0598a42e5136	8 hours ago	403MB
handsonsecurity/seed-elgg	original	b5a8ba77f28a	5 months ago	403MB
mysql	8.0.22	d4c3cafb11d5	5 years ago	545MB
handsonsecurity/seed-server	apache-php	2365d0ed3ad9	5 years ago	261MB

```
[03/23/26]seed@VM:~/Labsetup$
```

应该会看到上述六个镜像，是你在完成作业 2 时安装的。下述操作默认在你已完成作业 2 的情况下进行。下面可以执行以下命令，加载本次作业所需镜像：

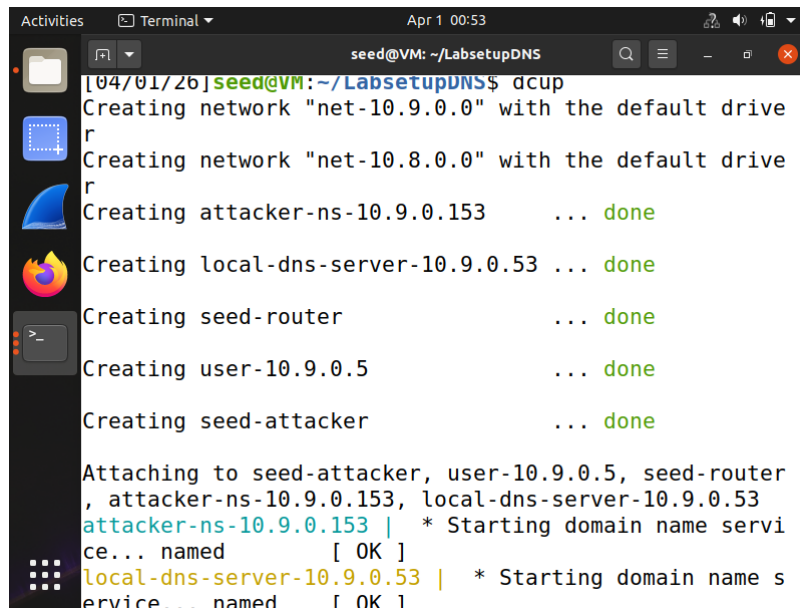
```
docker load -i dns-lab-delta.tar.xz
```

加载完成后运行 `dcbuild` 指令建立容器镜像，若建立成功，最后会看到 `Successfully built` 等信息。之后再运行 `docker images` 指令查看已安装的镜像如下：

```
[03/30/26]seed@VM:~/LabsetupDNS$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
seed-attacker-ns	latest	b303600c5d11	4 seconds ago	185MB
seed-user	latest	02b367ec14d6	5 seconds ago	264MB
seed-local-dns-server	latest	40eba821369a	28 seconds ago	185MB
seed-image-attacker-csrf	latest	524589161c9d	7 days ago	261MB
seed-image-mysql-csrf	latest	b951064dd350	7 days ago	545MB
seed-image-www-csrf	latest	379d16287d91	7 days ago	403MB
handsonsecurity/seed-elgg	original	b5a8ba77f28a	5 months ago	403MB
mysql	8.0.22	d4c3cafb11d5	5 years ago	545MB
handsonsecurity/seed-server	bind	bbf95098dacf	5 years ago	185MB
handsonsecurity/seed-server	apache-php	2365d0ed3ad9	5 years ago	261MB
handsonsecurity/seed-ubuntu	large	cecb04fbf1dd	5 years ago	264MB

接下来可以继续执行 `dcup` 指令启动服务器。成功效果如下图所示：



```

[04/01/26]seed@VM: ~/LabsetupDNS$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating network "net-10.8.0.0" with the default driver
Creating attacker-ns-10.9.0.153 ... done
Creating local-dns-server-10.9.0.53 ... done
Creating seed-router ... done
Creating user-10.9.0.5 ... done
Creating seed-attacker ... done

Attaching to seed-attacker, user-10.9.0.5, seed-router
, attacker-ns-10.9.0.153, local-dns-server-10.9.0.53
attacker-ns-10.9.0.153 | * Starting domain name servi
ce... named [ OK ]
local-dns-server-10.9.0.53 | * Starting domain name s
ervice... named [ OK ]

```

注意：和作业 2 相同，为使服务器继续运行，你需要留着这个窗口继续运行（不能关闭）。如果你需要关闭服务器，需要在 LabsetupDNS 文件夹打开另一个终端窗口并运行 **dcdown** 指令。这个指令运行完毕后，你可以看到两个终端窗口均输出停止信息并弹出命令提示符，表明服务器正常关闭。后续需要重新启动服务器时，在 LabsetupDNS 文件夹打开终端窗口并运行 **dcup** 指令即可，不必再 **dcbuild**。

2.1 容器设置和命令

在本次作业中，你需要在不同的角色（攻击者、用户、DNS 服务器）之间切换。这部分在实验手册 2.1 节有简单说明，但没有详细描述，很多同学也不了解如何在 Docker 环境下工作，在此给出具体说明。

在执行 **dcup** 指令后，本次作业中的攻击者、用户、DNS 服务器开始以 Docker 容器的方式在后台运行，可以理解为它们都有自己独立的 Linux 系统，作为虚拟机中的虚拟机存在。为了作为用户发送 DNS 请求、作为 DNS 服务器清空缓存，必须“进入”具体的容器，相当于在用户的命令行窗口或是 DNS 服务器的终端执行指令。

进入容器的常用方法是先用 **dockps** 查看正在运行的所有 Docker 容器的 ID，如下图所示：

```

[04/01/26]seed@VM: ~/LabsetupDNS$ dockps
2803be8912b3 seed-attacker
3fdb03c75511 user-10.9.0.5
92ab3c4fec79 seed-router
1fc19b7a22c5 local-dns-server-10.9.0.53
ab67ece697ab attacker-ns-10.9.0.153
[04/01/26]seed@VM: ~/LabsetupDNS$

```

可见攻击者、用户、本地 DNS 服务器等容器正在运行。然后可以使用 **docksh container_ID** 进入该容器的命令行模式，其中容器 ID 需要自行替换，只要前几位唯一匹配即可，如下图所示：

```

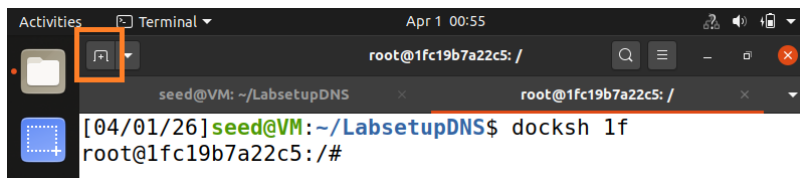
3fdb03c75511 user-10.9.0.5
92ab3c4fec79 seed-router
1fc19b7a22c5 local-dns-server-10.9.0.53
ab67ece697ab attacker-ns-10.9.0.153
[04/01/26]seed@VM: ~/LabsetupDNS$ docksh 3f
root@3fdb03c75511:/#

```

可见执行 **docksh 3f** 后，因为前缀为 3f 的容器只有 user 一个，已进入用户的命令行模式，相当于在虚拟的用户电脑上打开了命令行窗口，命令提示符也变为 **root@container_ID** 的形式。需要退出容器时，输入 **exit** 并按回车即可返回。实验手册 2.2 和 2.3 均为说明内容，不再赘述。

2.3 DNS 配置总结

本节的**本地 DNS 服务器**小节中，提到了本作业中会多次执行的命令：清空 DNS 缓存 `rndc flush`。此命令需要在本地 DNS 服务器容器中执行。需注意，你可以打开多个终端标签页，每个标签页进入一个容器，无需不断退出一个容器再进入另一个。如下图所示，可以点击橙色框内按钮打开新的终端标签页。



2.4 测试 DNS 设置

这部分的指令都应当在用户容器内执行。

3 攻击任务

3.1 任务 1：直接向用户伪造响应

实验手册要求你编写一个 Python 脚本完成攻击，这个脚本的源代码框架已在 4 **实验指导** 中给出，也可以在 LabsetupDNS 文件夹里的 volumes 文件夹里找到。有了框架，可以在不了解 Python 编程语言的情况下完成作业。但有一点需要注意：Python 使用缩进来定义代码块，请确保同一代码块的缩进量一致。如果缩进不同（比如 if 内的一行语句前面没有空格），Python 解释器会认为这行语句在 if 之外，导致运行结果不同，这和 C/C++/Java 是完全不一样的。正因如此，Python 不需要分号。

任务 1 无需框架中的权威部分和附加部分，但框架里最后的接口名部分需要解释，见下图粗体部分。

```
#!/usr/bin/env python3
from scapy.all import *
import sys

NS_NAME = "example.com"

def spoof_dns(pkt):
    if (DNS in pkt and NS_NAME in pkt[DNS].qd.qname.decode('utf-8')):
        print(pkt.sprintf("{DNS: %IP.src% --> %IP.dst%: %DNS.id%}"))

        ip = IP(...)           # Create an IP object
        udp = UDP(...)         # Create a UDP object
        Anssec = DNSRR(...)    # Create an answer record
        dns = DNS(...)         # Create a DNS object
        spoofpkt = ip/udp/dns  # Assemble the spoofed DNS packet
        send(spoofpkt)

myFilter = "..."           # Set the filter
pkt=sniff(iface='br-43d947d991eb', filter=myFilter, prn=spoof_dns)
```

实验手册里说“需要注意的是，在上述代码中，`iface` 参数中的值应替换为您实际的网络接口名，这是连接到 `10.9.0.0/24` 网络的接口。”。实验手册没有提及如何查看这个接口。你可以在 Home 目录里打开终端窗口，输入 `ifconfig` 命令并执行，这是用于配置和显示网络接口参数的命令。运行此命令的效果如下：

```
[03/31/26]seed@VM:~/LabsetupDNS$ ifconfig
br-c01a0c44b31c: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.9.0.1 netmask 255.255.255.0 broadcast 10.9.0.255
    inet6 fe80::42:28ff:febf:a557 prefixlen 64 scopeid 0x20<link>
    ether 02:42:28:cf:a5:57 txqueuelen 0 (Ethernet)
    RX packets 24 bytes 1068 (1.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 74 bytes 10527 (10.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

br-d5227fdd766c: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.8.0.1 netmask 255.255.255.0 broadcast 10.8.0.255
    inet6 fe80::42:36ff:feaa:f1a5 prefixlen 64 scopeid 0x20<link>
    ether 02:42:36:aa:f1:a5 txqueuelen 0 (Ethernet)
    RX packets 20 bytes 1112 (1.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 63 bytes 9325 (9.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

你需要找到对应 IP 地址段为 10.9.0.x 的，以 br-开头的接口名（图里涂黑部分），用它替换 Python 源代码里的 `iface` 参数。可以看到红线部分显示这个接口名对应的是 10.9.0.x 地址段。

注意：这个以 br-开头的接口名是动态分配的。每次你重启虚拟机或重启 Docker 网络，该名称都会改变，需要重新运行 `ifconfig`，查看接口名并替换源代码里的参数。

在完成其他所需修改后，可以开始执行 Python 脚本了。这个脚本作用于整个虚拟机，无须在容器内执行，在 volumes 文件夹里打开新终端窗口，用 `python3` 指令可以执行脚本，例如：

```
python3 dns_sniff_spoof.py
```

但你会发现这样直接运行会遇到 `PermissionError: [Errno 1] Operation not permitted` 错误，这是因为实验中使用的 `Scapy` 库需要调用原始接口（Raw Sockets）来绕过标准网络协议，而只有有 `Root` 权限的用户才有权限这么做。因此，需要以 `Root` 身份运行脚本：

```
sudo python3 dns_sniff_spoof.py
```

其中 `sudo` 是 `super user do` 的简称，指以超级用户（root）的身份执行命令，类似 Windows 系统中以管理员身份运行。需要注意，不能滥用 `sudo` 命令。脚本开始运行后，同样不能关闭它所在的终端窗口，否则会停止运行。脚本运行期间，可以在另一个窗口进入用户容器，执行 `dig www.example.com` 指令，观察脚本运行与否的结果差别。需要结束脚本运行时，选中它所在的窗口，按 `Ctrl+C` 终止即可。最后，每次测试之前，按需求在服务器容器内执行清空 DNS 缓存的指令。

3.2 - 3.5 任务 2 至任务 5

这部分任务可以合并到一个脚本执行。

4 实验指导

此章节没有需要完成的任务。

5 提交作业

请在作业 3 模板（Homework_3_Template.docx）上完成，**生成 pdf 文件并提交**。不要提交 .docx 文件。

作业中需要提交多张截图，请用虚拟机的截图功能截图。请勿手机拍屏。

本次作业没有其他需要提交的文件。